

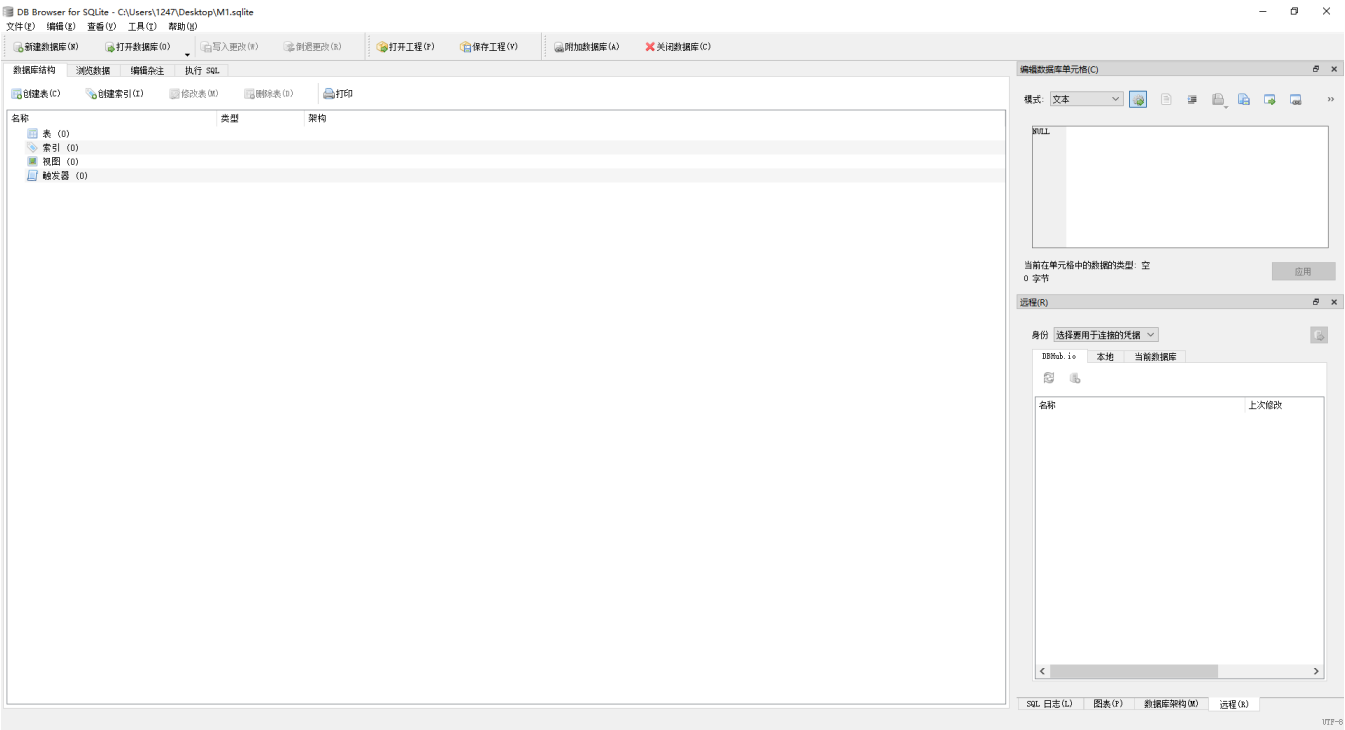
开发前准备

构建数据库

Step 1：使用DB Browser创建新的数据库

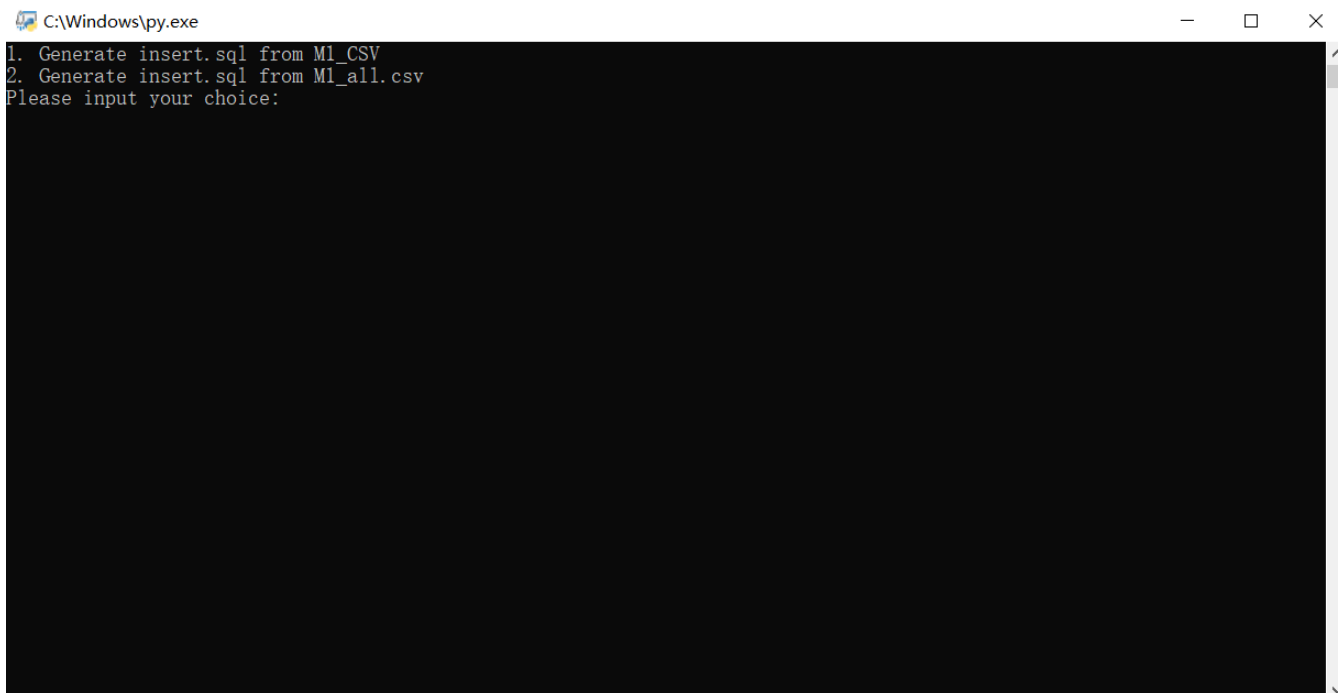
DB Browser安装包路径在 (Your Project Dir)/DatabaseOrm/db browser 中。

打开DB Browser(SQLcipher)，选择新建数据库（后缀为.sqlite），无需添加表。



Step 2：生成insert.sql（需要python环境）

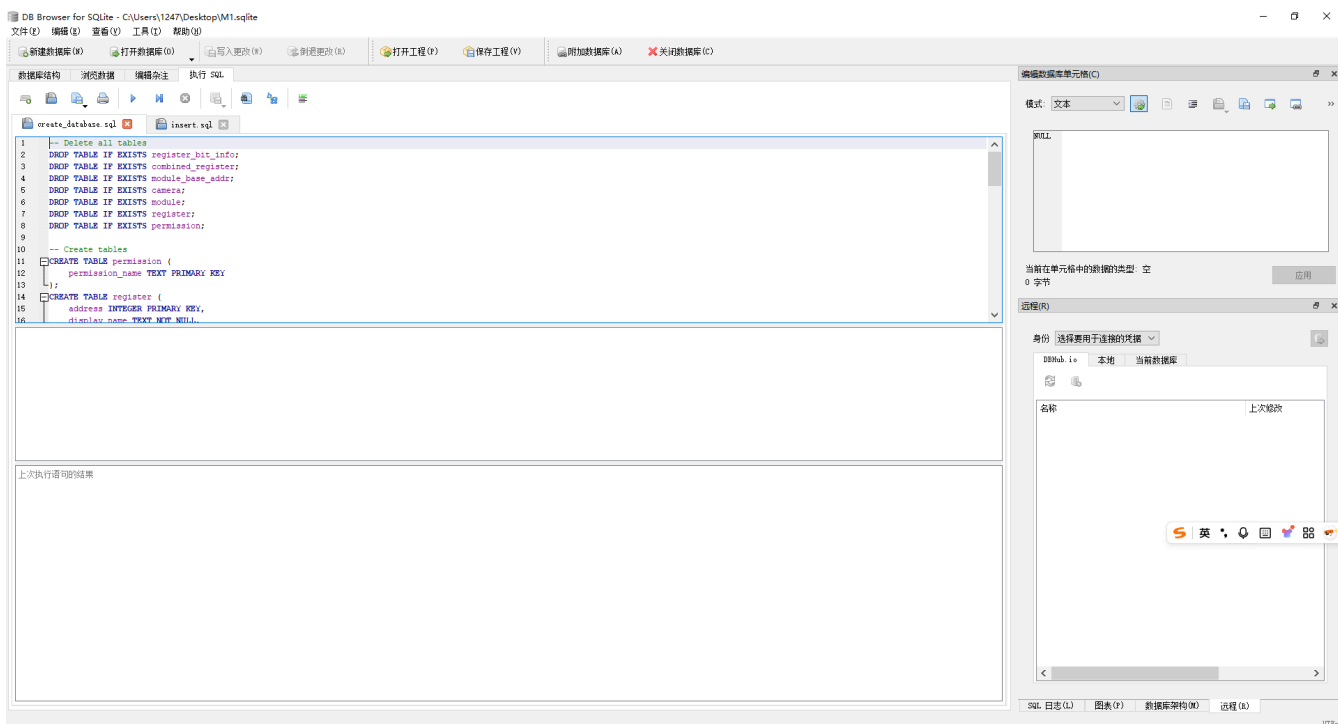
运行 (Your Project Dir)/DatabaseOrm/db/origin/combine_csv.py 文件



在此界面输入1，即通过M1_CSV目录中的csv文件来生成insert.sql，并替换到 (Your Project Dir)/DatabaseOrm/db/origin 路径。

Step 3：执行sql文件

切到 执行SQL 界面，选定sql文件执行 create_database.sql 与 insert.sql。sql文件目录位置为 (Your Project Dir)/DatabaseOrm/db/origin。

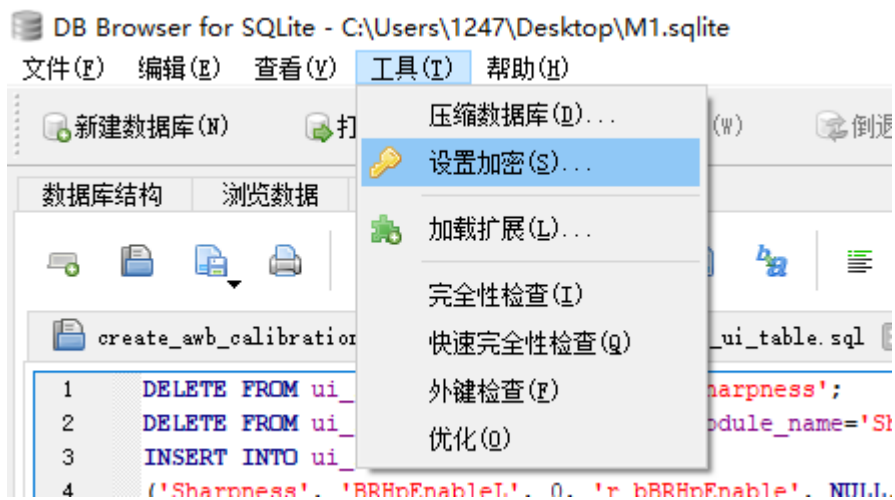


执行这两个sql语句文件后，需要继续执行ui_sql中相关的文件。文件前缀有两种（create/insert），需要先运行create，后运行insert。文件目录位置为

(Your Project Dir)/DatabaseOrm/db/origin/ui_sql。

Step 4: 加密

点击 工具-设置加密，设置密码为"Smartsens23"。设置完成后保存数据库即可。



Step 5: 替换

将sqlite文件重命名为M1.sqlite，并替换掉 (Your Project Dir)/DatabaseOrm/db 下的M1.sqlite文件，以及生成后 exe文件路径下的M1.sqlite文件。

开发

I. 添加页面

页面

ISPTuningTool的页面数在代码实现中与数据库中 ui_module 表相关。添加子tab可以在 insert_module.sql 中添加一个tuple。添加页面后再开发控件部分即可

```
1 INSERT INTO
2 ui_module (module_name, permission_name)
3 VALUES
4 ('ImgFmt', 'Administrator'),
5 ('AEC', 'Administrator'),
6 ('AWB', 'Administrator'),
7 ('DNS', 'Administrator'),
8 ('Sharpness', 'Administrator'),
9 ('DPC', 'Administrator'),
10 ('CAC', 'Administrator'),
11 ('Combine', 'Administrator'),
12 ('TM', 'Administrator'),
13 ('CCM', 'Administrator'),
14 ('Gamma', 'Administrator'),
15 ('GammaGain', 'Administrator'),
16 ('Contrast', 'Administrator'),
17 ('Saturation', 'Administrator'),
18 ('PRECCM', 'Administrator'),
19 ('LSC', 'Administrator'),
20 ('GBalance', 'Administrator'),
```

```
21 | ('RGBIR', 'Administrator'),
22 | ('NewPage', 'Administrator'); #新的页面
```

II. 建立控件与寄存器的对应关系

1. 在sql文件中添加控件ID与combined_register的tuple

找到控件对应页面的.sql文件（如AEC界面，对应sql文件为 `insert_aec.sql`），按照表 `ui_info` 以及 `ui_info_regmodule`（详情见文档数据库设计.md）的内容插入tuple。

2. 代码中控件与combined_register的映射

控件与 `combined_register` 的映射依靠变量 `combinedRegisterMap` 实现。

```
1 | QMap<QString, std::vector<CombinedRegister>> combinedRegisterMap;
```

其中key为控件的label值（部分控件可能不一样，对于RGB/IR帧分开调的寄存器，需要去除名字中的"*"号，并在label后增加"`_RGB_AB / _IR_AB`"，且仅在0x30031f03值为1时生效）。

3. 将寄存器记录至代码

可通过调用函数 `recordCombineRegbyUIifo` 来实现。

```
1 | void QTabWndComm::recordCombineRegbyUIifo(QString moudle_name, QString uiname,
    std::shared_ptr<int> cameraId);
```

参数列表：

参数	对应的DB表 : 列
moudle_name	combined_register : module_name
uiname	ui_info : control_name
cameraId	camera : camera_id

此操作可以将控件对应的 `combined_register` 写入变量 `combinedRegisterMap` 中。部分页面可能有较为特殊的情况（不同module的控件名称相同），此时可以重写/重载该函数，比如将key值改为"子页面_控件名称"的形式。

4. 记录寄存器对应的子tab

可通过实现并调用函数 `combinedRegisterSubPageRecording` 来实现。部分需要RGB/IR帧分开调的寄存器控件需要实现并调用 `getCombinedRegister` 函数。

一般页面的 `combinedRegisterSubPageRecording` 函数写法如下：

```
1 | std::vector<CombinedRegister> combinedRegisterSubPageRecording(QString strModuleName,
    QString strUiName, std::shared_ptr<int> ptrCamera, QString strSubPage)
2 | {
3 |     std::vector<UiInfo> uiInfo;
```

```

4 // qDebug() << strModuleName << ", " << strUiName;
5 uiInfo= DatabaseOrm::getUiInfoByModuleNameAndControlName(moduleName,
strUiName.toStdString());
6 std::vector<CombinedRegister> combinevector;
7 for (auto i : uiInfo) {
8     CombinedRegister combineReg
9         = DatabaseOrm::getCombinedRegisterByModuleNameAndCameraIdAndCombinedName(
10         strModuleName.toStdString(), ptrCamera, *(i.combinedName));
11     combinevector.push_back(combineReg);
12 }
13
14 for (auto reg : combinevector)
15 {
16     auto reBitInfo = DatabaseOrm::getRegisterBitInfoByCombinedRegister(reg);
17     for (int i = 0; i < reBitInfo.size(); i++)
18     {
19         regAddrModule.insert(reBitInfo[i].registerAddr);
20     }
21     std::vector<uint32_t>().swap(registerMoudle);
22     for (auto iter : regAddrModule)
23     {
24         registerMoudle.emplace_back(iter);
25     }
26     // qDebug() << reg.combinedName.c_str() << ", " << strSubPage;
27     setRegisterSubPage(reg, strSubPage);
28 }
29
30 return combinevector;
31 }

```

`getCombinedRegister` 函数写法如下:

```

1 std::vector<CombinedRegister> getCombinedRegister(QString strModuleName, QString
strUiName, std::shared_ptr<int> ptrCamera)
2 {
3     std::vector<UiInfo> uiInfo;
4     // qDebug() << strModuleName << ", " << strUiName;
5     uiInfo= DatabaseOrm::getUiInfoByModuleNameAndControlName(moduleName,
strUiName.toStdString());
6     std::vector<CombinedRegister> combinevector;
7     for (auto i : uiInfo) {
8         CombinedRegister combineReg
9             = DatabaseOrm::getCombinedRegisterByModuleNameAndCameraIdAndCombinedName(
10             strModuleName.toStdString(), ptrCamera, *(i.combinedName));
11         combinevector.push_back(combineReg);
12     }
13     return combinevector;
14 }

```

对于特殊情况可能需要特殊处理（比如AEC界面，可参照代码的qaectab.cpp）。

如果控件的寄存器拥有分开调RGB/IR的寄存器，则需要调用 `getCombinedRegister` 函数获取RGB/IR对应的寄存器，再调用 `setRegisterTypeMap` 函数。

```
1 void QTabWndComm::setRegisterTypeMap(std::vector<CombinedRegister> regVector,
    std::vector<CombinedRegister> regVectorRgb, std::vector<CombinedRegister> regVectorIr);
```

可参考qawbtab.cpp的实例进行实现。

III. 页面下方表格的寄存器信息显示

鼠标点击到控件的label时，需要在下方表格显示寄存器的信息。

实现方法：使用connect槽函数连接控件与recvEdit函数

```
1 connect(edit,&QLabelEditGroup::releaseLabel,this,[=]() {recvEdit(edit->Label());});
2 connect(box,&QLabelCheckBox::release,this,[=]() {recvEdit(box->Label());});
```

部分需要RGB/IR帧分开调的情况，可以写一个新的函数来实现。

比如 `recvEditTarget` 的实现，用于应对RGB/IR帧分开调的情况，核心逻辑先试用 `clearCurrentRigester` 清楚表格中的寄存器，再使用数据库相关函数（详情见文档数据库设计.md）拿到combined_register，之后再调用 `updateRegisterTable` 函数将寄存器逐个插入表格中：

```
1 void QAECTab::recvEditTarget(QString str, QString strModule)
2 {
3     clearCurrentRigester();
4
5     int valTemp = 0;
6     QUsbCommunication::getInstance().readBurst(0x30031f03, (unsigned char*)&valTemp,
    ADDRESS_LENGTH, VALUE_LENGTH);
7
8     std::vector<CombinedRegister> re;
9     std::string strSuffix = strRgbIrMode == "RGB" ? "RGB_AB" : "IR_AB";
10    if (valTemp == 0x01)
11    {
12        re = combinedRegisterMap.value(strModule + "_" + str + " " +
    strSuffix.c_str());
13    }
14    else
15    {
16        re = combinedRegisterMap.value(strModule + "_" + str);
17    }
18
19    updateRegisterTable(re);
20 }
```

IV. 更新UI界面信息

一般需要在注册表下方的slider/checkbox/edit控件，或者点击read按钮时触发。一般界面会用 `updateUIValue` 函数。

```
1 template <typename T>
2 void updateUIValue(T& vector, const QString &extraname="");
```

该函数会将变量 `vector` 中包括的控件中的值更新到界面上。对于不同界面可以采用不同的实现方法，例如imgfmt界面的情况：

```
1 template <typename T>
2 void updateUIValue(T& vector, const QString &extraname="")
3 {
4     for(auto item:vector){
5         uint32_t value=0;
6         auto reg=combinedRegisterMap.value(item->Label()+extraname);
7         for(int i=0;i<reg.size();i++){
8             readCombinedRegisterValue(reg.at(i), value);
9             if (item->Label() == "width" || item->Label() == "Height")
10                 item->setValue(QString("%1").arg(value));
11             else
12                 item->setValue(QString("0x%1").arg(QString::number(value,16).toUpper().rightJustified(VALUE_
13                 HEX_INFO, '0'))));
14         }
15     }
```

核心逻辑是获取每个控件的CombinedRegister的Value，再将value显示到对应控件上。